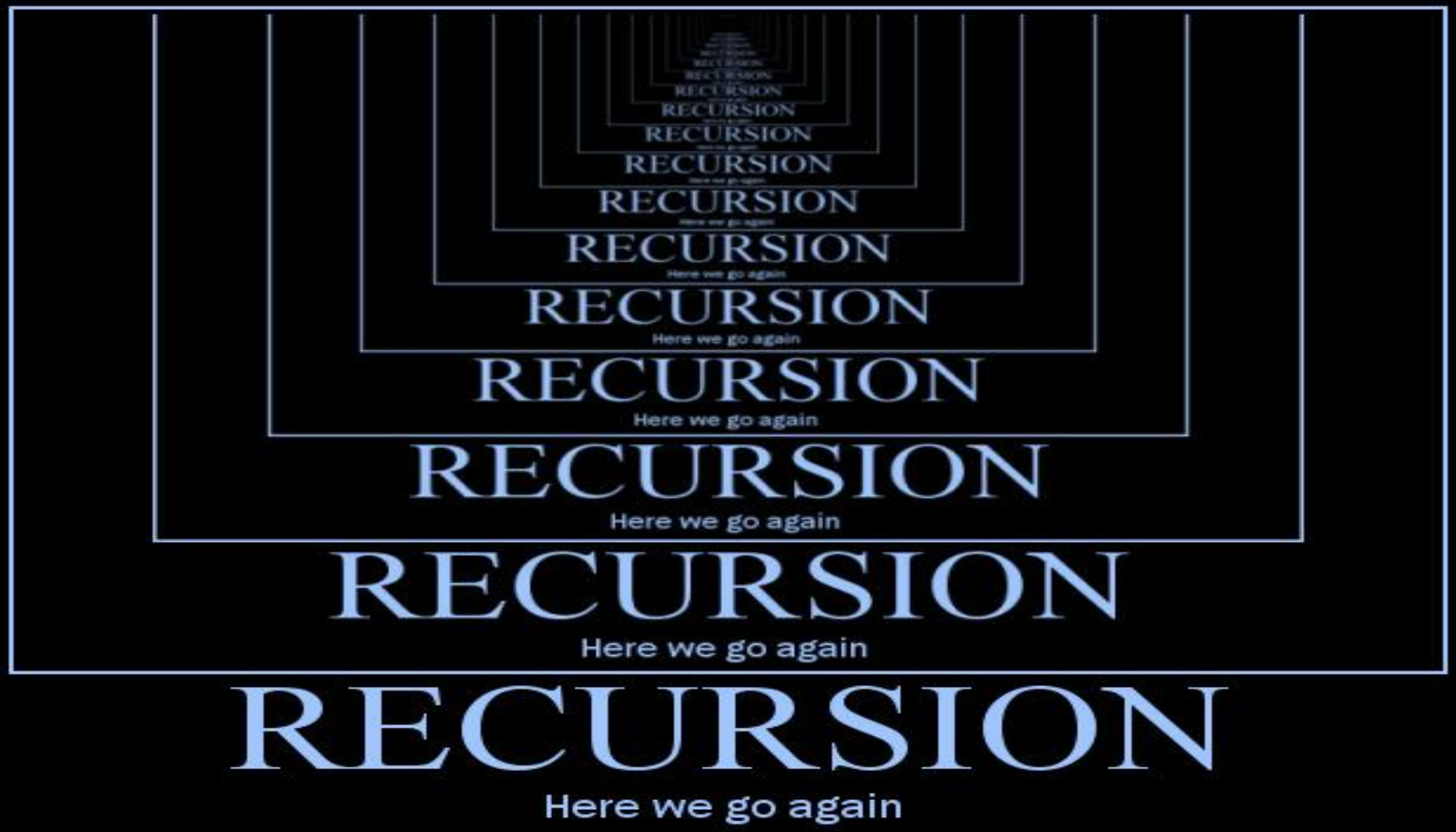


Java Programming

CPT111 – Week 12

Erick Purwanto

Xi'an Jiaotong-Liverpool University

西交利物浦大学

CPT111 Java Programming

Week 12

Recursion

Outline

- Examples of Recursion
  - Factorial
  - Recursion God vs Recursive Calls
  - Handshake Problem
- Recursion Recipe
  - Base Case
  - Recursive Step
  - with Helper Method
- More examples
  - Fibonacci Number, The Twelve Days of Christmas, Subsequences

Learning Outcomes and Reading

On successful completion of this module, students are expected to:

A. Understand and appreciate the principles of using object oriented programming techniques for the construction of professional robust, maintainable programs deployed to meet real world business goals;
B. Design, write, compile, test, debug and execute Java programs using an integrated development environment (IDE);
C. Be able to develop programs that the user can interact with either through graphical representation, or text based menus;
D. Use Java programming to represent, display, and manipulate data as objects while being cognizant of associated risk/safety for safe operation of computing and information systems.
E. Have an awareness of essential legal, social, ethical & professional issues related to software development

- Reading from optional textbooks:
  - [IJPDS] Chapter 18
  - [AJ] Chapter 11

Coursework Feedback

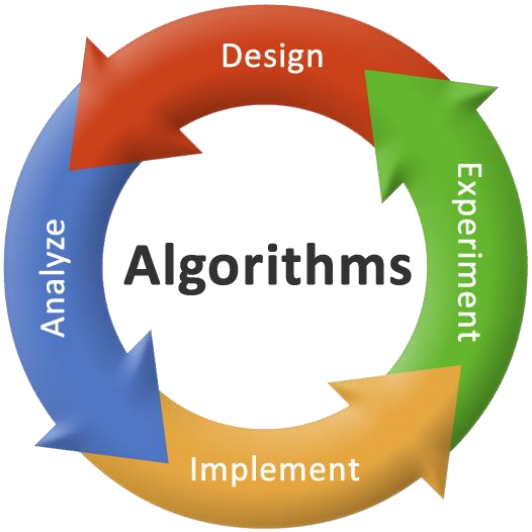
```java
// CW1 #9.2 DISTANCE TO WITH EXCEPTION
public double distanceTo(EuclideanPoint otherPoint) {
    double deltaX = otherPoint.x - this.x;
        double deltaY = otherPoint.y - this.y;
        double distance=Math.sqrt(deltaX * deltaX + deltaY * deltaY);
        if (distance==0){
            throw new IllegalArgumentException("Zero distance detected");
        }
        return distance;


}
```

```java
// CW1 #9.2 DISTANCE TO WITH EXCEPTION
public double distanceTo(EuclideanPoint otherPoint) {
    double length;
    length = Math.sqrt(Math.pow(this.x - otherPoint.x,2) + Math.pow(this.y - otherPoint.y,2));
    if(length==0)
    throw DistanceToException("Zero distance detected");
    return length;
}
```

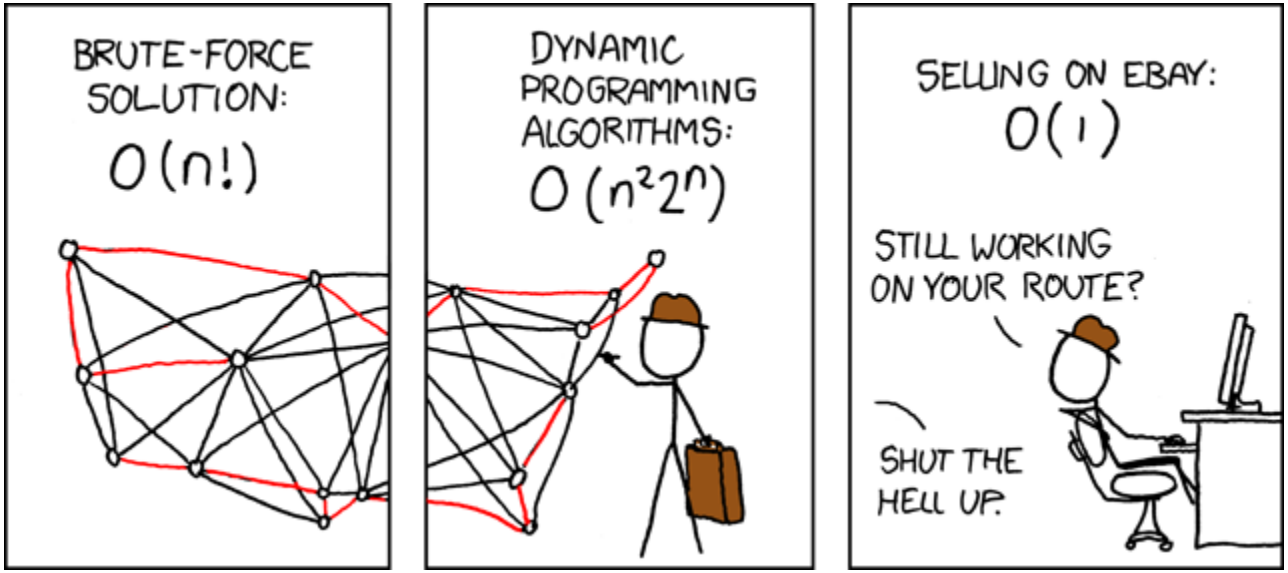
- Coursework 1 Week 9 Feedback to Students:
  - need to check the type of exceptions
  - make sure an object of exceptions is thrown

Algorithms Design Techniques



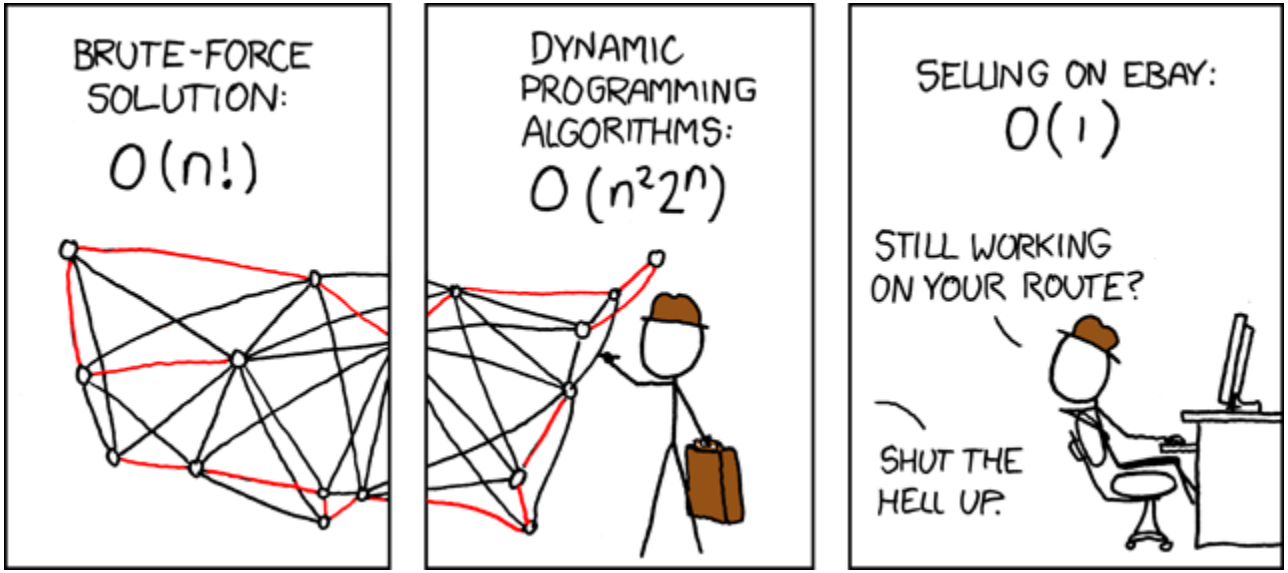
- Recursion is one of the tools in designing algorithm
- Transition from coding to problem solving
- Designing **fast** algorithm
  - Divide-and-Conquer := Divide + Delegate (Recursion) + Combine
  - Dynamic Programming := Recursion + Memoization
  - will more learn in *CPT204 Advanced OOP*

Travelling Salesman Problem



- Asks what is the shortest tour that visits *all* n cities
- Brute-force solution: try all tours
- How many possible tours are there ? n ! possible tours

Factorial



- Factorial `fact(n)` of a number n >= `0` is defined as follows
  - `fact(0) = 1`
  - `fact(n) = n * n-1 * n-2 * ··· * 3 * 2 * 1`

In-Class Quiz 1: Factorial Number (1)

What is factorial number of 4, `fact(4)` ?

a) `fact(4) = 4`

b) `fact(4) = 12`

c) `fact(4) = 24`

In-Class Quiz 2: Factorial Number (2)

What is factorial number of 5, `fact(5)` ?

a) `fact(5) = 5`

b) `fact(5) = 24`

c) `fact(5) = 120`

In-Class Quiz 3: Factorial Number (3)

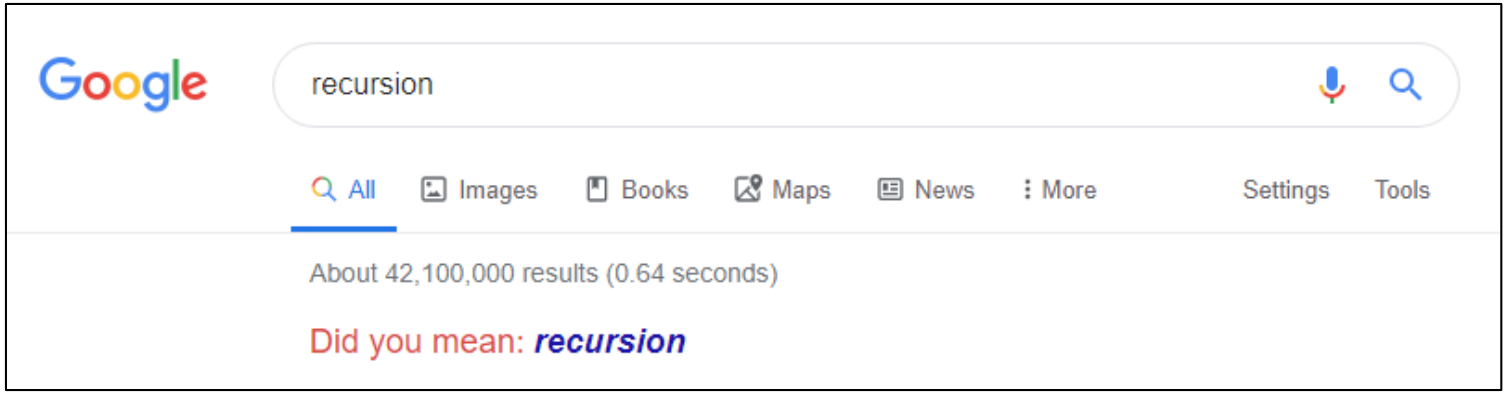
What is factorial number of n, `fact(n)` ?

a) `fact(n) = n * (n-1)`

b) `fact(n) = fact(n-1) * n`

c) `fact(n) = fact(n-1) * (n-1)`

Recursive Factorial



- Factorial `fact(n)` of a number `n >= 0` is *recursively* defined as follows
  - `fact(0) = 1`
  - `fact(n) = fact(n-1) * n`

Recursion Code

```
public static long fact(int n) {
    if (n == 0)
        return 1;
    else
        return fact(n-1) * n;
}
```

```
public static void main(String[] args) {
    long answer = fact(5);
}
```

Recursion God ?!

```
public static long fact(int n) {
    if (n == 0)
        return 1;
    else
        return n * fact(n-1);
}
```

Recursion God

- *Recursion God* will tell you the answer of `fact(n-1)` !!
  - Magic ?!

Inception

Have you guys watched Inception ?

Warning ! Major SPOILER ahead !

Inception World : Sharing Dream inside a Dream ! (1)

real world

↓

dream no. 1

↓

dream no. 2

↓

dream no. 3

↓

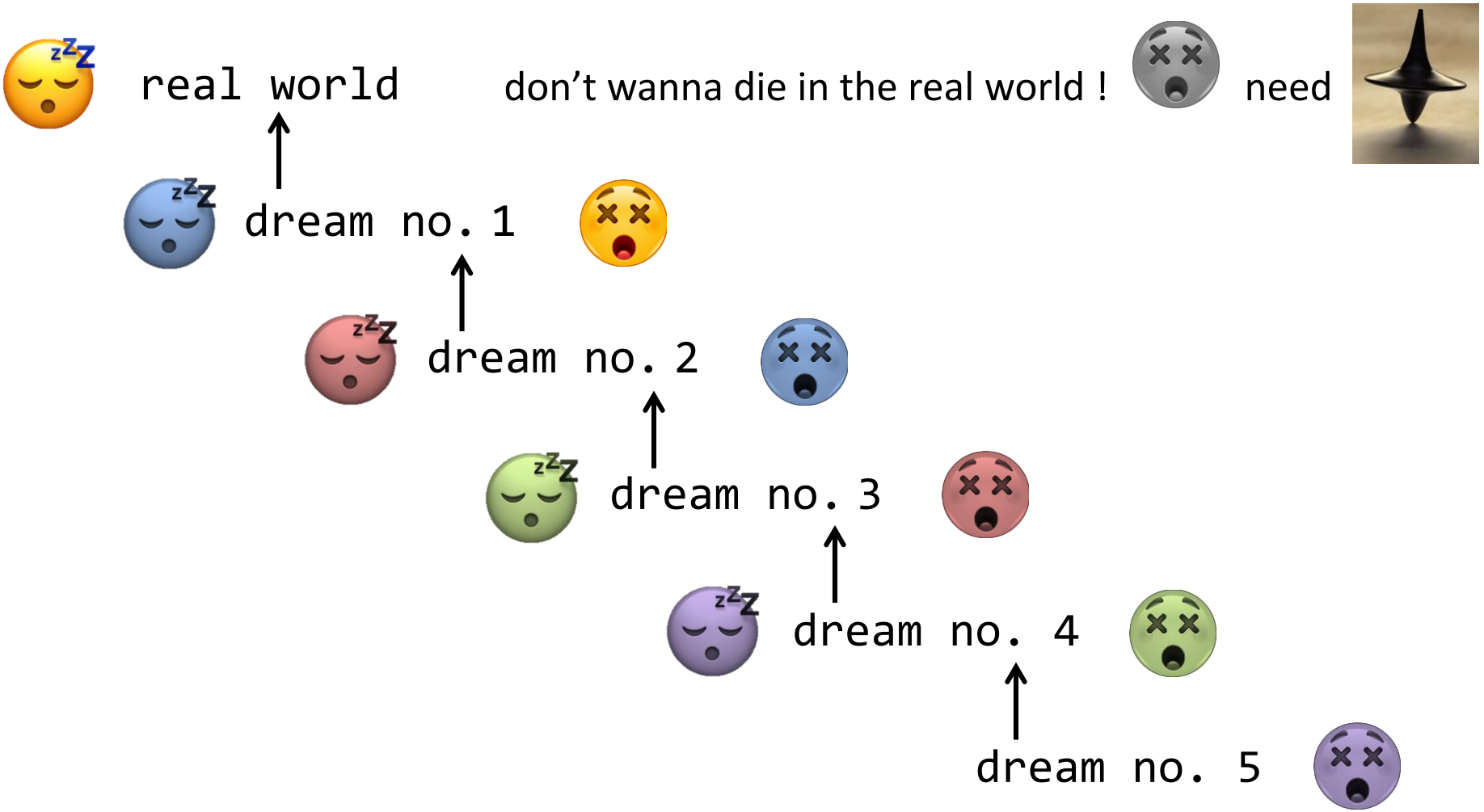
dream no. 4

↓

dream no. 5

Dream Machine

Inception World : Sharing Dream inside a Dream ! (2)



Factorial Recursive Calls (1)

```
fact(5)
  ↓
return 5 * fact(4)
            ↓
        return 4 * fact(3)
                    ↓
                return 3 * fact(2)
                            ↓
                        return 2 * fact(1)
                                    ↓
                                return 1 * fact(0)
                                            ↓
                                            1
```

Factorial Recursive Calls (2)

```
fact(5) ⟶ 120

return 5 * fact(4) ⟶ 24

        return 4 * fact(3) ⟶ 6

                return 3 * fact(2) ⟶ 2

                        return 2 * fact(1) ⟶ 1

                                return 1 * fact(0)

                                        1
```

The Handshake Problem

- There are n people in a room
- If each person shake hands *once* with every other person, what is the total number of handshakes `handshake(n)` ?

In-Class Quiz 4: Handshake Problem : Base Case

What is the base case of the Handshake Problem ?

a) `handshake(2) = 0`

b) `handshake(2) = 1`

c) `handshake(2) = 2`

In-Class Quiz 5: Handshake Problem : Recursive Step

A third person comes. What is the number of handshakes now ?

a) `handshake(3) = handshake(2) + 3`

b) `handshake(3) = handshake(2) + 2`

c) `handshake(3) = handshake(2) + 1`

Complete The Handshake Problem Code !

```
public static int handshake(int n) {



}
```

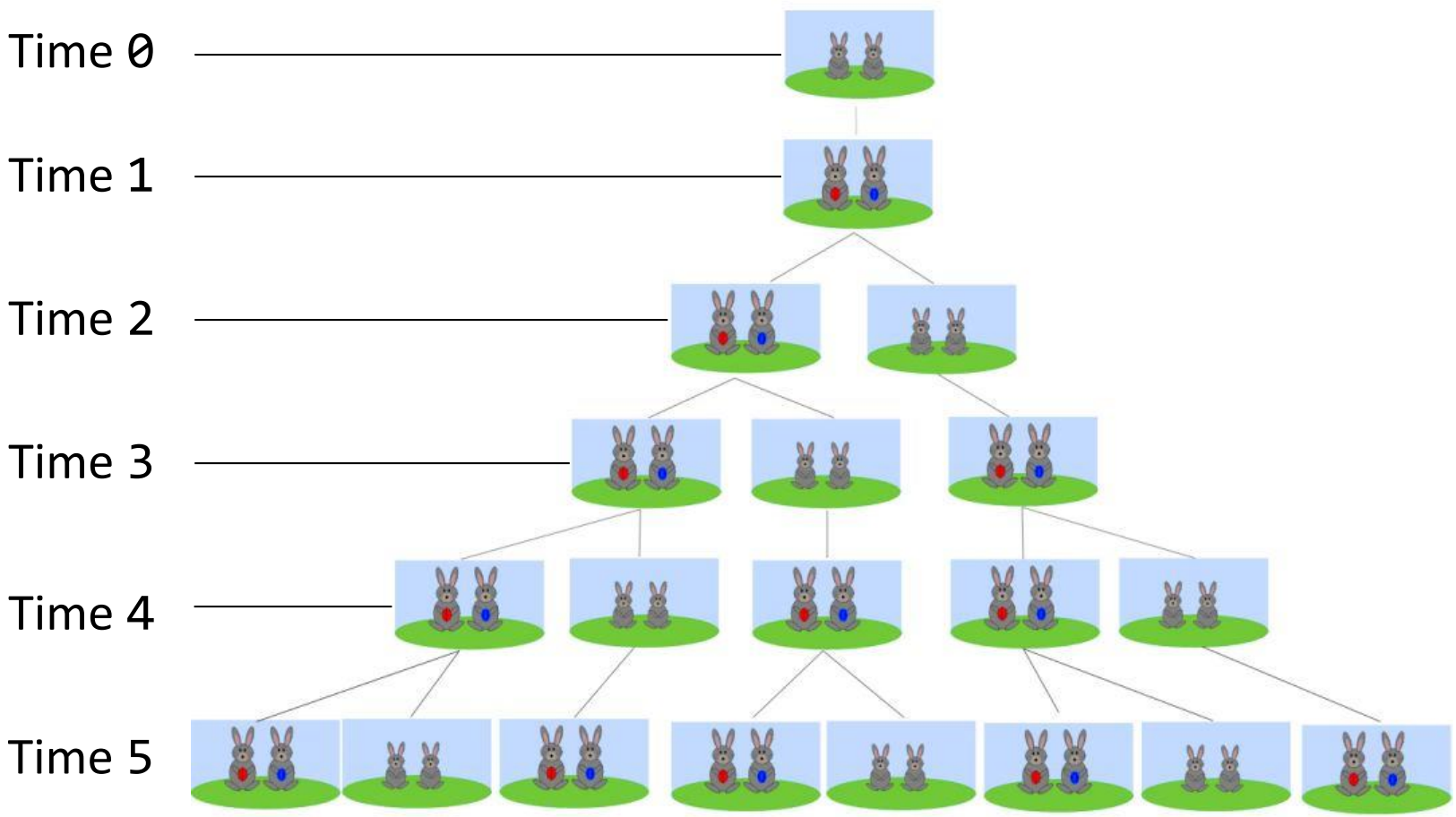
Recipe for Recursion

The problem can be **reduced** into smaller and easier subproblems

<=> the solution to a problem **depends** on the solutions to smaller instances of the same problem

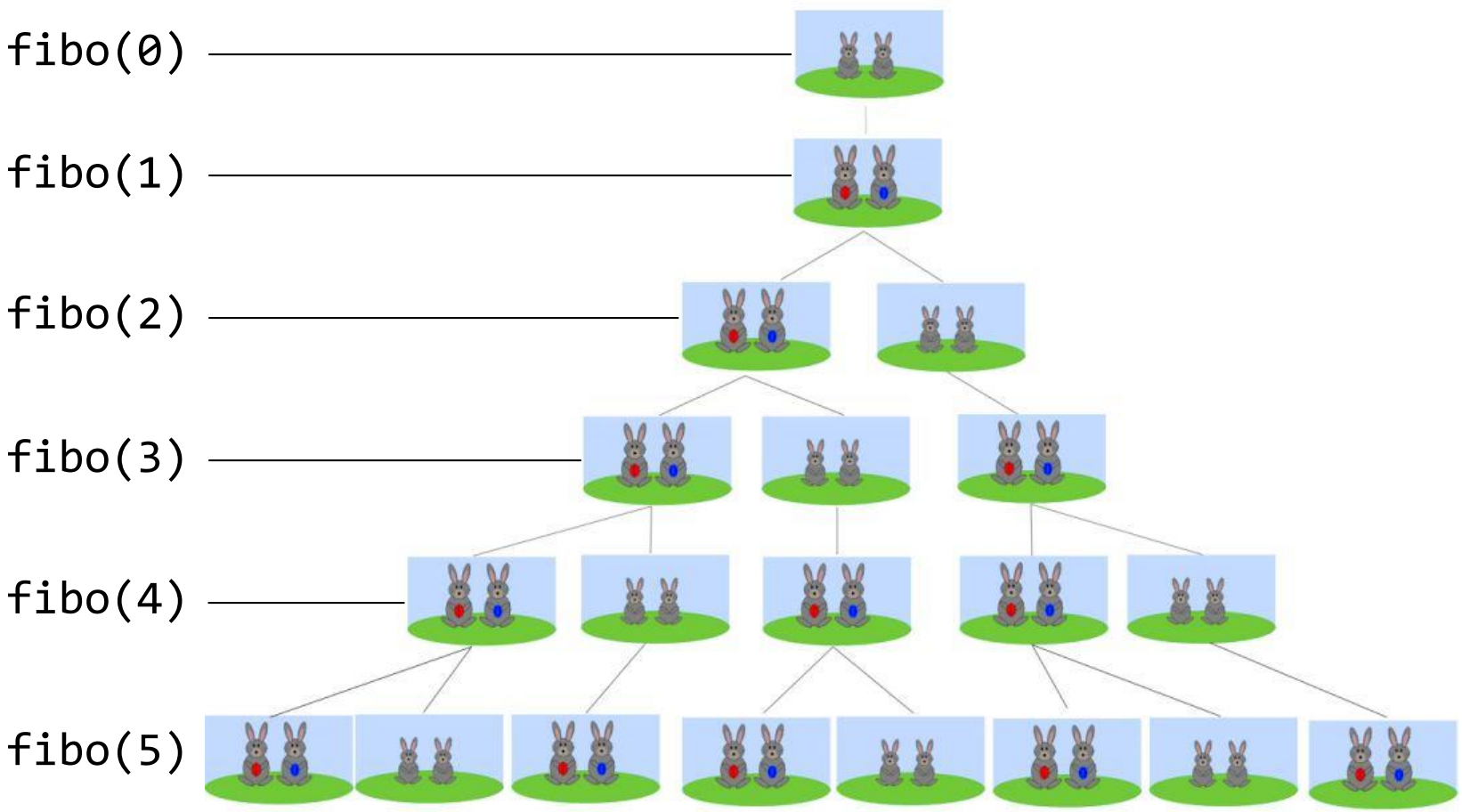
1. Base Case :
   - Find the simplest instance that can be **directly** solved
2. Recursive Step :
   - Try a couple of cases to see the recursive **relation** among instances
   - Formulate generally for n,
     trust Recursion God to solve the smaller instances
   - Make sure to reach the base case

Rabbit Breeding Problem



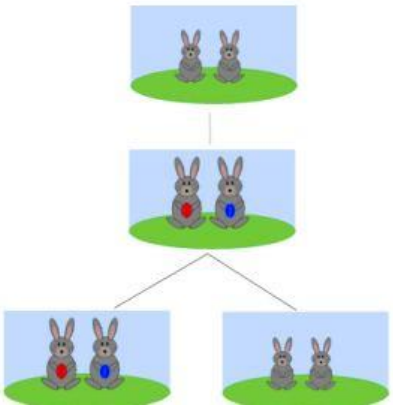
- Assume a pair of rabbits take one time period to mature and breed, always conceive a male female pair, never die
- How many pairs of rabbits after time period Time n ?

Fibonacci Number



- What is the Fibonacci number `fibo(n)` ?

In-Class Quiz 6: Fibonacci Number : Base Case



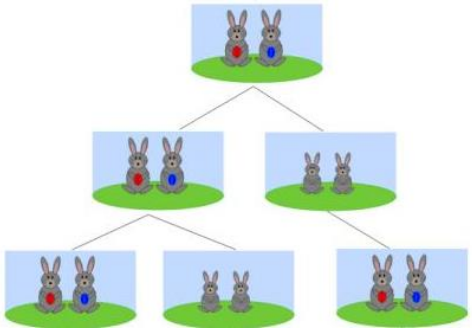
What is the base case of the Fibonacci Number ?

a) `fibo(0) = 1`

b) `fibo(1) = 1`

c) `fibo(0) = 1 and fibo(1) = 1`

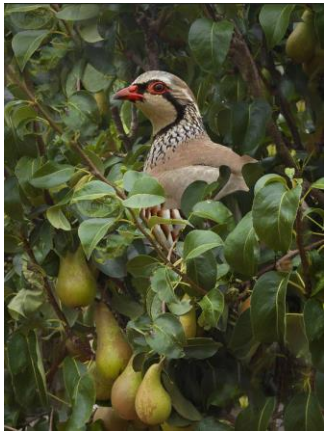
Complete the Fibonacci Number Code!



```
public static int fibo(int n) {

}
```

The Twelve Days of Christmas



- Let's sing together!
- On the first day of Christmas
  My true love gave to me
  A partridge in a pear tree.

- On the second day of Christmas
  My true love gave to me
  Two turtle doves,
  And a partridge in a pear tree.

- On the third day of Christmas
  My true love gave to me
  Three French hens,
  Two turtle doves,
  And a partridge in a pear tree.

The Twelve Days of Christmas

- On the fourth day of Christmas
  My true love gave to me
  Four calling birds,
  Three French hens,
  Two turtle doves
  And a partridge in a pear tree.

- On the fifth day of Christmas
  My true love gave to me
  Five golden rings,
  Four calling birds,
  Three French hens,
  Two turtle doves
  And a partridge in a pear tree ...

- Use recursion to compute *how many* gifts your true love gave to you on the *n*th day of Christmas!
  And, *total* gifts up to *n*th days of Christmas!

The Twelve Days of Christmas

- Test cases:

n	#gifts on day n	#total gifts on day n
1	1	1
2	3	4
3	6	10
4	10	20
5	15	35
12	78	364
400	80200	10746800

- Complete the code for the Twelve Days of Christmas!

Recursion with Helper Methods

- For the Subsequences Problem, instead of solving it with *direct* recursive implementation,

```
public static String subseq(String word)
```

it is easier to use additional *Helper Method*

```
private static String subseqHelper(String
                                   partialSubseq, String word)
```

- Helper method introduces another parameter(s) to give you more flexibility in doing (more creative) recursion calls

Subsequences Problem

- Given a word consisting only of letters A-Z or a-z
- Return all subsequences of word, separated by commas, where a subsequence is a string of letters found in word in the same order that they appear in word.

```
subsequences("abc") -> "abc,ab,bc,ac,a,b,c,"
```

- Note the trailing comma preceding the empty string `""`, which is also a valid subsequence

Subsequences with Helper Methods (1)

```
private static String subseqHelper(String
                                   partialSubseq, String word)
```

- Idea: we build up a partial subsequence using the initial letters of the word
  - then use recursive calls to complete that partial subsequence using the remaining letters of the word
- For example, word = "abc"
  - First, select "a" to be in the partial subsequence, and recursively extend it with all subsequences of "bc"
  - Then, use "" as the partial subsequence, and again recursively extend it with all subsequences of "bc"

Subsequences with Helper Methods (2)

```
subsequences("abc") -> "abc,ab,bc,ac,a,b,c,"
```

- In other words, recursively build up subsequences that, either use the first letter `a` or not…

Subsequences Code

```
private static String subseqHelper(String
                                   partialSubseq, String word) {
   if (word.isEmpty())
      // base case
      return partialSubseq;
   else
      // recursive step
      return subseqHelper(partialSubseq + word.charAt(0),
                                          word.substring(1))
            + ","
            + subseqHelper(partialSubseq,
                                          word.substring(1));
}
```

Notes for Recursive + Helper Methods

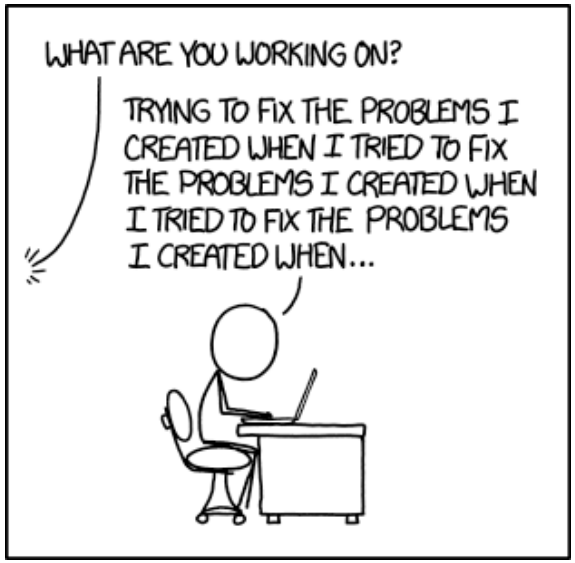
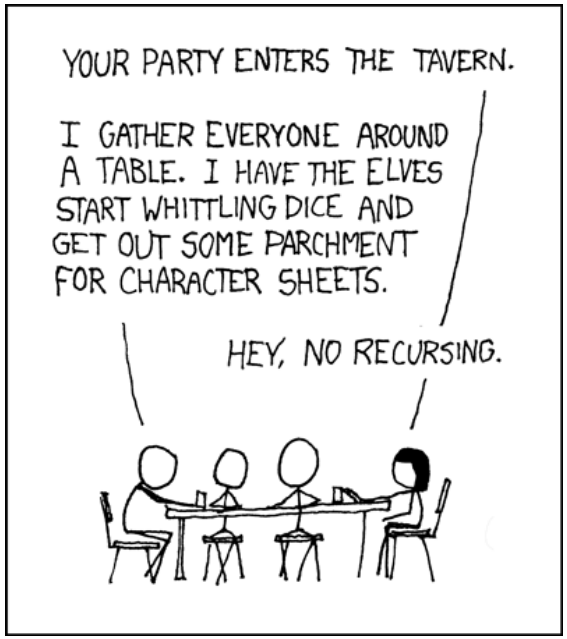
- Don’t forget to implement the original problem

```
public static String subseq(String word)
    return subseqHelper("", word);
}
```

- You can use the same method name for your helper methods

- Hide the helper methods, don’t expose it to the client

Tasks of Week 12

- Lab 12
- Exercise and CW1 Week 12
- Watch Inception (again) !

Thank you for your attention !